

Assignment 4

Robot Framework Test Automation: Syntax, Data-Driven Testing, Custom Libraries, Assertions, Setup/Teardown, Tags and Reporting

Overview

This document provides a complete, working Robot Framework solution for all seven task groups. Every listing below is taken directly from the project files, so the code in this document is identical to the code that is executed. The web tests target the public practice site **the-internet.herokuapp.com** (login page) and the API test targets **jsonplaceholder.typicode.com**; both are stable, free services intended for automation practice.

Project Structure

```
robot_assignment4/
+-- requirements.txt
+-- data/
|   +-- login_data.csv           (external test data)
+-- libraries/
|   +-- MathLibrary.py           (custom Python keywords)
+-- resources/
|   +-- common.resource          (shared variables and keywords)
+-- tests/
|   +-- 01_basic_syntax.robot     (Task 1)
|   +-- 02_variables_data_driven.robot (Task 2)
|   +-- 03_custom_keywords_libraries.robot (Task 3)
|   +-- 04_assertions_verification.robot (Task 4)
|   +-- 05_setup_teardown.robot   (Task 5)
|   +-- 06_tags_execution.robot   (Task 6)
|   +-- 07_reports_logs.robot     (Task 7)
```

Environment Setup

```
pip install -r requirements.txt
```

The requirements file installs Robot Framework, SeleniumLibrary (browser automation), RequestsLibrary (API testing) and Pabot (parallel execution). Google Chrome must be installed; Selenium Manager downloads the matching driver automatically. Add `--variable BROWSER:headlesschrome` to any run to execute without a visible browser window.

```
robotframework>=7.0
robotframework-seleniumlibrary>=6.3
robotframework-requests>=0.9.6
robotframework-pabot>=4.0
```

1. Basic Syntax and Keywords

A test file has *Settings*, *Variables* and *Test Cases* sections; keywords and arguments are separated by four spaces. The suite below opens a browser and navigates to a URL, fills form fields with **Input Text**, and confirms elements exist with **Page Should Contain Element**. A test teardown closes the browser after every test.

tests/01_basic_syntax.robot

```
*** Settings ***
Documentation      Task 1 - Basic syntax and keywords.
Library            SeleniumLibrary
Test Teardown      Close Browser

*** Variables ***
${URL}             https://the-internet.herokuapp.com/login
${BROWSER}         chrome
```

```

*** Test Cases ***
Open Browser And Navigate To URL
[Documentation]    Open a web browser and navigate to a specific URL.
Open Browser      ${URL}      ${BROWSER}
Location Should Be    ${URL}
Title Should Be      The Internet

Fill Form Field Using Input Text
[Documentation]    Use the Input Text keyword to fill in a form field.
Open Browser      ${URL}      ${BROWSER}
Input Text        id=username    tomsmith
Input Text        id=password    SuperSecretPassword!
Textfield Value Should Be    id=username    tomsmith

Verify Element Is Present
[Documentation]    Verify a specific element using Page Should Contain Element.
Open Browser      ${URL}      ${BROWSER}
Page Should Contain Element    id=username
Page Should Contain Element    id=password
Page Should Contain Element    css:button[type='submit']

```

Run:

```
robot --outputdir results/task1 tests/01_basic_syntax.robot
```

Expected outcome: 3 tests, 3 passed. The URL and page title are confirmed, the username field holds *tomsmith*, and the username, password and submit elements are all found.

2. Variables and Data-Driven Automation

Username and password are stored once as variables (`${USERNAME}`, `${PASSWORD}`) in the shared resource file and reused by keywords. For data-driven testing, the CSV file below is read with **Get File**, the header is skipped using **Split To Lines** (start index 1), and a **FOR** loop runs the same login steps for each row. The third column holds the message expected for that row.

resources/common.resource

```

*** Settings ***
Documentation      Shared variables and keywords used by the test suites.
Library            SeleniumLibrary

*** Variables ***
${BASE_URL}       https://the-internet.herokuapp.com
${LOGIN_URL}      ${BASE_URL}/login
${BROWSER}        chrome
${USERNAME}       tomsmith
${PASSWORD}       SuperSecretPassword!

*** Keywords ***
Login With Credentials
[Arguments]       ${username}    ${password}
Input Text        id=username    ${username}
Input Password    id=password    ${password}
Click Button      css:button[type='submit']

Open Browser And Login
Open Browser      ${LOGIN_URL}    ${BROWSER}
Maximize Browser Window
Login With Credentials    ${USERNAME}    ${PASSWORD}
Element Should Contain    id=flash    You logged into a secure area!

Logout And Close Browser
Run Keyword And Ignore Error    Click Element    css:a.button.secondary
Run Keyword And Ignore Error
...    Element Should Contain    id=flash    You logged out of the secure area!
Close Browser

```

data/login_data.csv

```
username,password,expected_message
tomsmith,SuperSecretPassword!,You logged into a secure area!
tomsmith,WrongPassword,Your password is invalid!
invaliduser,SuperSecretPassword!,Your username is invalid!
```

tests/02_variables_data_driven.robot

```
*** Settings ***
Documentation      Task 2 - Variables and data-driven automation.
Resource           ../resources/common.resource
Library           OperatingSystem
Library           String
Test Setup        Open Browser    ${LOGIN_URL}    ${BROWSER}
Test Teardown     Close Browser

*** Variables ***
${DATA_FILE}      ${CURDIR}${/}..${/}data${/}login_data.csv

*** Test Cases ***
Login Using Variables
    [Documentation]  Credentials are stored in variables (see common.resource).
    Login With Credentials    ${USERNAME}    ${PASSWORD}
    Element Should Contain    id=flash    You logged into a secure area!

Data Driven Login From CSV File
    [Documentation]  Read every row of login_data.csv and run the same steps.
    ${content}=      Get File    ${DATA_FILE}
    @{rows}=         Split To Lines    ${content}    1
    ${count}=        Get Length    ${rows}
    Log    Data rows found in CSV file: ${count}
    FOR    ${row}    IN    @{rows}
        @{cols}=    Split String    ${row}    ,
        Log    Testing user '${cols}[0]' - expecting '${cols}[2]'
        Login With Credentials    ${cols}[0]    ${cols}[1]
        Element Should Contain    id=flash    ${cols}[2]
        Delete All Cookies
        Go To    ${LOGIN_URL}
    END
```

Run:

```
robot --outputdir results/task2 tests/02_variables_data_driven.robot
```

Expected outcome: 2 tests, 2 passed. The CSV contains 3 data rows: the valid login shows the success message, a wrong password shows *Your password is invalid!*, and an unknown user shows *Your username is invalid!*.

3. Custom Keywords and Libraries

Any Python module can be used as a Robot Framework library: each public function becomes a keyword whose name is the function name in Title Case (add_numbers becomes **Add Numbers**). The library below is imported by path. The suite also uses **BuiltIn** keywords (**Catenate**, **Get Length**, **Evaluate**, **Convert To Integer**) for string and math operations.

libraries/MathLibrary.py

```
"""Custom Robot Framework library written in Python."""

def _to_number(value):
    """Convert a string or number to int (preferred) or float."""
    try:
        return int(value)
    except (TypeError, ValueError):
        return float(value)

def add_numbers(first, second):
    """Return the sum of two numbers.
```

```

Example:
| ${total}= | Add Numbers | 5 | 7 |
"""
return _to_number(first) + _to_number(second)

def multiply_numbers(first, second):
    """Return the product of two numbers."""
    return _to_number(first) * _to_number(second)

```

tests/03_custom_keywords_libraries.robot

```

*** Settings ***
Documentation      Task 3 - Custom Python keyword and BuiltIn library usage.
Library            ../libraries/MathLibrary.py

*** Test Cases ***
Sum Of Two Integers Using Custom Keyword
    ${result}=      Add Numbers      5      7
    Should Be Equal As Integers      ${result}      12

Sum Of Two Decimals Using Custom Keyword
    ${result}=      Add Numbers      2.5      3.5
    Should Be Equal As Numbers       ${result}      6.0

String Manipulation Using BuiltIn
    ${text}=        Catenate          SEPARATOR=${SPACE}      Robot      Framework
    ${length}=      Get Length        ${text}
    ${upper}=       Evaluate          "${text}".upper()
    ${words}=       Evaluate          len("${text}".split())
    Should Be Equal    ${text}        Robot Framework
    Should Be Equal As Integers    ${length}      15
    Should Be Equal    ${upper}      ROBOT FRAMEWORK
    Should Be Equal As Integers    ${words}       2

Math Operations Using BuiltIn
    ${product}=     Evaluate          12 * 5
    ${root}=        Evaluate          math.sqrt(144)      modules=math
    ${number}=      Convert To Integer      42
    Should Be Equal As Integers    ${product}      60
    Should Be Equal As Numbers     ${root}        12
    Should Be Equal As Integers    ${number}      42

```

Run:

```
robot --outputdir results/task3 tests/03_custom_keywords_libraries.robot
```

Expected outcome: 4 tests, 4 passed. $5 + 7 = 12$, $2.5 + 3.5 = 6.0$, "Robot Framework" has 15 characters and 2 words (upper case: ROBOT FRAMEWORK), $12 \times 5 = 60$ and the square root of 144 is 12.

4. Assertions and Verification

The first test performs an action (login), stores the *expected* heading, reads the *actual* heading with **Get Text** and compares them with **Should Be Equal**. The API test sends a GET request with RequestsLibrary and verifies the response with **Should Be Equal As Strings** (status code, post id and user id).

tests/04_assertions_verification.robot

```

*** Settings ***
Documentation      Task 4 - Assertions and verification (UI, keyword and API).
Resource          ../resources/common.resource
Library           RequestsLibrary
Library           ../libraries/MathLibrary.py

*** Variables ***
${API_URL}        https://jsonplaceholder.typicode.com/posts/1

*** Test Cases ***
Login Action Matches Expected Heading

```

```

[Documentation]    Perform an action, then compare expected and actual results.
[Teardown]        Close Browser
Open Browser      ${LOGIN_URL}    ${BROWSER}
Login With Credentials    ${USERNAME}    ${PASSWORD}
${expected}=      Set Variable    Secure Area
${actual}=         Get Text      css:h2
Should Be Equal    ${actual}      ${expected}

```

```

Addition Result Matches Expected Value
${actual}=      Add Numbers    10    15
Should Be Equal As Integers    ${actual}    25

```

```

API Response Is Verified
[Documentation]    Verify an API response with Should Be Equal As Strings.
${response}=      GET    ${API_URL}
Should Be Equal As Strings    ${response.status_code}    200
${body}=           Set Variable    ${response.json()}
Should Be Equal As Strings    ${body}[id]    1
Should Be Equal As Strings    ${body}[userId]    1

```

Run:

```
robot --outputdir results/task4 tests/04_assertions_verification.robot
```

Expected outcome: 3 tests, 3 passed. The heading is *Secure Area*, $10 + 15 = 25$, and the API returns status 200 with id = 1 and userId = 1.

5. Test Setup and Teardown

Test Setup runs *Open Browser And Login* before every test, and **Test Teardown** runs *Logout And Close Browser* after every test (even when the test fails). Both keywords are defined in `common.resource` (shown in Task 2). The logout step is wrapped in **Run Keyword And Ignore Error** so that the browser is always closed and no resources are left behind.

tests/05_setup_teardown.robot

```

*** Settings ***
Documentation    Task 5 - Common test setup (open browser + login) and teardown.
Resource        ../resources/common.resource
Test Setup      Open Browser And Login
Test Teardown    Logout And Close Browser

*** Test Cases ***
Secure Area Is Displayed After Login
    Location Should Contain    /secure
    Page Should Contain        Secure Area

Logout Button Is Available
    Page Should Contain Element    css:a.button.secondary

```

Run:

```
robot --outputdir results/task5 tests/05_setup_teardown.robot
```

Expected outcome: 2 tests, 2 passed. In `log.html` each test shows a Setup entry (login) and a Teardown entry (logout and close).

6. Tags and Test Execution

Tags are assigned per test with `[Tags]` and for the whole suite with `Test Tags` (every test in this file also carries the *demo* tag). Tests are then selected from the command line with `--include` and `--exclude`.

tests/06_tags_execution.robot

```

*** Settings ***
Documentation    Task 6 - Tags and selective / parallel execution.
Resource        ../resources/common.resource
Library         ../libraries/MathLibrary.py

```

```

Test Tags          demo

*** Test Cases ***
Login Page Loads
  [Tags]    smoke    login
  [Teardown]  Close Browser
  Open Browser    ${LOGIN_URL}    ${BROWSER}
  Page Should Contain Element    id=username

Valid Login Works
  [Tags]    regression    login
  [Teardown]  Close Browser
  Open Browser    ${LOGIN_URL}    ${BROWSER}
  Login With Credentials    ${USERNAME}    ${PASSWORD}
  Element Should Contain    id=flash    You logged into a secure area!

Invalid Login Is Rejected
  [Tags]    regression    negative    login
  [Teardown]  Close Browser
  Open Browser    ${LOGIN_URL}    ${BROWSER}
  Login With Credentials    ${USERNAME}    WrongPassword
  Element Should Contain    id=flash    Your password is invalid!

Addition Sanity Check
  [Tags]    smoke    math
  ${total}=    Add Numbers    1    2
  Should Be Equal As Integers    ${total}    3

```

Selective execution:

```

robot --include smoke --outputdir results/smoke tests/06_tags_execution.robot
robot --exclude regression --outputdir results/no_regression tests/06_tags_execution.robot
robot --include loginANDregression --outputdir results/login_regression tests/06_tags_execution.robot
robot --include mathORnegative --outputdir results/math_or_negative tests/06_tags_execution.robot

```

Command option	Tests selected	Count
--include smoke	Login Page Loads, Addition Sanity Check	2
--exclude regression	Login Page Loads, Addition Sanity Check	2
--include loginANDregression	Valid Login Works, Invalid Login Is Rejected	2
--include mathORnegative	Addition Sanity Check, Invalid Login Is Rejected	2

Parallel execution:

The core robot command has no `--processes` option; that option belongs to **Pabot** (`robotframework-pabot`), the standard parallel runner for Robot Framework. It distributes the suites across worker processes and merges the results into a single report.

```
pabot --processes 4 --outputdir results/parallel tests
```

Expected outcome: the tag-filtered runs execute only the tests listed in the table above, all passing. The Pabot run executes all 20 tests from the seven suites across 4 processes and produces one combined `log.html` and `report.html`.

7. Reports and Logs

Robot Framework always writes `output.xml`, `log.html` and `report.html`. Custom messages are written with the **Log** keyword using levels (INFO, DEBUG, WARN); **Log To Console** prints to the terminal and **Log Many** writes several values at once. DEBUG messages appear only when the log level is lowered with `--loglevel DEBUG`; WARN messages are highlighted in yellow in the report and listed under Test Execution Errors.

tests/07_reports_logs.robot

```

*** Settings ***
Documentation    Task 7 - Reports and custom log messages.

```

```

Library                ../libraries/MathLibrary.py

*** Test Cases ***
Custom Log Messages For Debugging
    Log    Test started - preparing input values    level=INFO
    ${first}=    Set Variable    8
    ${second}=    Set Variable    4
    Log    Inputs are first=${first} and second=${second}    level=DEBUG
    Log To Console    Calculating the sum of ${first} and ${second}
    ${total}=    Add Numbers    ${first}    ${second}
    Log    Calculated total is ${total}
    Log Many    Expected: 12    Actual: ${total}
    Should Be Equal As Integers    ${total}    12
    Log    Assertion passed - test finished    level=INFO

Warning Level Message Is Highlighted In Report
    Log    This message is shown as a warning in the log file    level=WARN
    ${product}=    Multiply Numbers    6    7
    Should Be Equal As Integers    ${product}    42

```

Run with an HTML report:

```

robot --loglevel DEBUG --outputdir results/report \
      --output output.xml --log log.html --report report.html \
      --reporttitle "Assignment 4 Test Report" --logtitle "Assignment 4 Test Log" \
      tests

```

Expected outcome: all suites are listed in report.html with per-test status, tags and elapsed time. log.html contains the custom INFO, DEBUG and WARN messages from suite 07, including the calculated total of 12 and the product 42. The console prints *Calculating the sum of 8 and 4*.

Results

Robot Framework and the internet-dependent services (browser, web application, public API) were not available in the environment used to prepare this document. The results are therefore reported in two clearly separated parts: what was **actually executed**, and what each Robot test is **expected** to produce when run with the commands above.

A. Checks executed while preparing this document

Check	Result	Detail
Add Numbers 5 + 7	PASS	returned 12
Add Numbers 2.5 + 3.5	PASS	returned 6.0
Add Numbers 10 + 15	PASS	returned 25
Add Numbers 1 + 2	PASS	returned 3
Add Numbers 8 + 4	PASS	returned 12
Multiply Numbers 6 x 7	PASS	returned 42
String: length	PASS	len=15
String: upper	PASS	ROBOT FRAMEWORK
String: word count	PASS	2 words
Math: 12 * 5	PASS	60
Math: sqrt(144)	PASS	12.0
CSV: data rows (header skipped)	PASS	3 rows
CSV: 3 columns per row	PASS	[3, 3, 3]
Syntax layout: resources/common.resource	PASS	sections present, no tabs
Syntax layout: tests/01_basic_syntax.robot	PASS	sections present, no tabs
Syntax layout: tests/02_variables_data_driven.robot	PASS	sections present, no tabs
Syntax layout: tests/03_custom_keywords_libraries.robot	PASS	sections present, no tabs
Syntax layout: tests/04_assertions_verification.robot	PASS	sections present, no tabs
Syntax layout: tests/05_setup_teardown.robot	PASS	sections present, no tabs
Syntax layout: tests/06_tags_execution.robot	PASS	sections present, no tabs
Syntax layout: tests/07_reports_logs.robot	PASS	sections present, no tabs

Summary: 21 checks executed, 21 passed, 0 failed. These confirm the custom Python keywords, the BuiltIn-style calculations, the CSV file structure (3 data rows, 3 columns) and the layout of every .robot and .resource file.

B. Expected outcome of every Robot test case

Suite	Test case	Key verification	Expected status
01	Open Browser And Navigate To URL	URL and title verified	PASS
01	Fill Form Field Using Input Text	Field value is tomsmith	PASS
01	Verify Element Is Present	3 elements found	PASS
02	Login Using Variables	Success message displayed	PASS
02	Data Driven Login From CSV File	3 of 3 rows behave as expected	PASS

Suite	Test case	Key verification	Expected status
03	Sum Of Two Integers Using Custom Keyword	$5 + 7 = 12$	PASS
03	Sum Of Two Decimals Using Custom Keyword	$2.5 + 3.5 = 6.0$	PASS
03	String Manipulation Using BuiltIn	15 characters, 2 words, upper-cased	PASS
03	Math Operations Using BuiltIn	60, 12 and 42	PASS
04	Login Action Matches Expected Heading	Secure Area	PASS
04	Addition Result Matches Expected Value	25	PASS
04	API Response Is Verified	Status 200, id 1, userId 1	PASS
05	Secure Area Is Displayed After Login	URL contains /secure	PASS
05	Logout Button Is Available	Logout element present	PASS
06	Login Page Loads	Tags: smoke, login, demo	PASS
06	Valid Login Works	Tags: regression, login, demo	PASS
06	Invalid Login Is Rejected	Password error message shown	PASS
06	Addition Sanity Check	$1 + 2 = 3$	PASS
07	Custom Log Messages For Debugging	Total 12, messages logged	PASS
07	Warning Level Message Is Highlighted In Report	$6 \times 7 = 42$, WARN logged	PASS

Expected total: 20 tests, 20 passed, 0 failed (7 suites).

C. Output files produced by a run

File	Description
report.html	High-level report: statistics by suite and tag, pass/fail status of each test
log.html	Detailed execution log with keyword-level steps, setup/teardown and custom log messages
output.xml	Machine-readable results used to generate the report and log

To reproduce the actual results, run `robot --outputdir results tests` from the project folder and open `results/report.html`.